

AI 데이터 분석 · 7일차

조건문과 반복문

프로그램의 흐름을 바꾸는 두 개의 도구

if · elif · else · 중첩 · for · range · 누적 변수 · enumerate

갈림길에서 고르고(조건), 같은 일을 되풀이한다(반복)

교안 04_02(조건문) · 05_01(for 반복문) · Practice · 수업 노트 정리

2026-07-29 AI 데이터 분석 교육 정리 (7일차)

학습자 Minho · 국립금오공과대학교 컴퓨터공학부

이 책을 읽는 법

지금까지 배운 코드는 전부 **위에서 아래로 한 번씩** 흘렸습니다. 오늘 배우는 두 도구가 그 흐름을 바꿉니다 — `if`는 갈림길에서 하나를 고르게 하고, `for`는 같은 일을 되풀이하게 합니다. 이 둘이 있어야 비로소 "수천 개 데이터에서 이상값만 골라내는" 코드를 쓸 수 있습니다.

조건문 `if·elif·else` **반복문** `for·range` **연산자** `and·or·not` **패턴** 누적 변수 **내장함수** `enumerate`

연결

3일차에 만든 참·거짓(`temp > 80`)이 오늘 드디어 쓰입니다. 그때는 `True/False`를 `print`로 보기만 했는데, 오늘은 그 값으로 **실제 행동을 결정**합니다. 6일차의 리스트도 반복문과 만나 진가를 발휘하고요.

목차

	장 / STEP	무엇을 배우나
CHAPTER 0	오늘의 지도 — 흐름을 바꾸는 두 도구	순차 실행 · 분기 · 반복
STEP 1	조건식 — <code>True/False</code> 만들기	비교 연산자 · 경계 표현(이상·초과)
STEP 2	<code>if</code> 와 <code>else</code>	콜론 · 들여쓰기 · 코드 블록
STEP 3	<code>elif</code> — 갈림길이 셋 이상	여러 갈래 · 순서의 함정
STEP 4	논리 연산자와 중첩 <code>if</code>	<code>and</code> · <code>or</code> · <code>not</code> · 범위 조건 · 중첩
STEP 5	<code>for</code> 와 <code>range</code>	기본 구조 · <code>range</code> 3형태 · 역순
STEP 6	누적 변수 패턴	합계 · 개수 · 평균 · 초기화 위치
STEP 7	<code>enumerate</code> 와 중첩 반복	인덱스+값 · 구구단 · 실행 횟수

CHAPTER 0

오늘의 지도 — 흐름을 바꾸는 두 도구

순차 실행에서 벗어나기

프로그램은 기본적으로 위에서 아래로 한 줄씩 순서대로 실행됩니다. 코드를 쓰는 순서가 곧 실행 순서죠. 그런데 현실의 일은 그렇게 단순하지 않습니다.

개념 흐름을 바꿔야 하는 순간 왜 조건문·반복문인가

정의 "항상 똑같이"가 아니라 "상황에 따라 다르게" 또는 "여러 번 되풀이"해야 할 때가 있습니다.

```
# 일상 비가 오면 우산을 챙기고, 안 오면 그냥 나간다
# 로그인 비밀번호가 맞으면 입장, 틀리면 재입력 요청
# 설비 온도가 정상 범위를 벗어나면 경고, 정상이면 통과

# 반복 측정값 100개를 하나씩 확인해 기준 초과 개수 세기
```

왜 쓰나 앞의 셋은 갈림길(분기), 마지막은 되풀이(반복)입니다. 각각 if와 for가 담당합니다.

세 가지 실행 흐름



흐름	하는 일	오늘 배울 것
순차 실행	코드를 위에서 아래로 한 번씩	(지금까지 해온 것)
분기	조건에 따라 실행할 코드를 고름	if · elif · else
반복	같은 코드를 N번 되풀이	for · range()

개념

두 도구의 공통 문법이 하나 있습니다 — **콜론(:)으로 줄을 끝내고, 그 아래를 들여쓰다**. `if`도 `for`도 똑같습니다. 콜론은 "여기서부터 이 구조에 속한다"는 신호이고, 들여쓰기는 "어디까지 속하는가"를 나타냅니다. 이 규칙 하나가 오늘 배울 모든 문법의 뼈대입니다.

STEP 1

조건식 — True/False 만들기

판단의 재료를 만드는 단계

if는 참·거짓을 보고 움직입니다. 그러니 먼저 참·거짓을 만들 줄 알아야 합니다. 3일차에 배운 비교 연산자가 바로 그 도구입니다.

연산자 비교 연산자 6종 조건식을 만드는 기호

정의 두 값을 견주어 참(True) 또는 거짓(False)을 냅니다. 결과는 항상 불리언(bool).

```
print(10 > 3)      # True
print(10 < 3)      # False
temp = 85

print(temp > 80)   # True   (85는 80보다 크다)
temp = 70
print(temp > 80)   # False  (70은 80보다 크지 않다)
```

```
▶ True
▶ False
▶ True
▶ False
```

왜 쓰나 if 뒤에 오는 것은 결국 이 조건식입니다. 조건식이 True면 실행, False면 건너뜁니다. 참·거짓은 True/False로 쓰며 대문자로 시작하고 따옴표는 없습니다.

응용 조건식 자체를 변수에 담아두면 코드가 훨씬 읽기 쉬워집니다. 조건이 복잡해질수록 효과가 큼니다.

```
temp = 85
is_hot = temp > 80      # 조건식 결과를 변수에
print(is_hot)           # True
print(type(is_hot))     # <class 'bool'>
```



```
if is_hot:              # 읽기 쉬운 조건
    print("온도 주의")
```

```
▶ True
▶ <class 'bool'>
▶ 온도 주의
```

경계 표현 — 이상·초과·이하·미만

조건을 정할 때 가장 자주 실수하는 곳이 ****경계값****입니다. 우리말과 기호를 정확히 짝지어 두세요.

우리말	기호	경계값	예시 (기준 80)
이상	<code>>=</code>	포함	80 포함 — 80도 해당
초과	<code>></code>	미포함	80 제외 — 81부터 해당
이하	<code><=</code>	포함	80 포함
미만	<code><</code>	미포함	79까지 해당

주의

실습 1에서 "19세 이상 성인"을 판정할 때 `age >= 19`와 `age > 19`는 결과가 다릅니다. 19를 넣어보면 바로 갈리죠. 조건을 만든 뒤엔 **반드시 경계값으로 테스트**하는 습관을 들이세요.

개념 조건 판단 설계 3단계 막막할 때의 순서

정의 조건문을 쓰기 전에 이 세 가지를 말로 정리하면 코드가 저절로 나옵니다.

```
# ① 상황 파악 — 판단할 값이 무엇인가?      → 측정 온도 temp
# ② 조건 정의 — 기준을 비교식으로 바꾸면?    → temp > 80
# ③ 결과 지정 — 참일 때 무엇을 하나?        → "경고" 출력
```

```
temp = 85
if temp > 80:
    print("경고")
```

▶ 경고

왜 쓰나 "어떻게 짜지?"에서 막힐 때 대부분은 ②번(기준을 비교식으로 옮기기)에서 막힙니다. 우리말 기준을 먼저 적고 위의 경계 표를 보며 기호로 옮기세요.

STEP 2

if 와 else

콜론과 들여쓰기가 전부다

조건문 if 문의 기본 구조 조건이 참일 때만 실행

정의 if 조건식: 아래에 들여쓴 코드는 조건이 참일 때만 실행됩니다. 들여쓰기 밖의 코드는 조건과 무관하게 항상 실행됩니다.

형태 if 조건식: 실행할 코드

```
temp = 85
if temp > 80:                # 만약 온도가 80을 넘으면
    print("온도 주의")       # 들여쓴 이 줄만 실행
    print("점검 필요")       # 같은 블록 → 역시 조건부
print("측정 종료")           # 들여쓰기 밖 → 항상 실행
```

- ▶ 온도 주의
- ▶ 점검 필요
- ▶ 측정 종료

왜 쓰나 콜론(:)은 "여기서부터 이 조건에 속하는 코드가 시작된다"는 신호이고, 들여쓴 한 덩어리를 ****코드 블록****이라 부릅니다. if·elif·else·for 모두 콜론으로 끝나는 공통 규칙입니다.

개념

같은 코드에서 temp = 70으로 바꾸면 조건이 거짓이라 들여쓴 두 줄은 건너뛰고 "측정 종료"만 출력됩니다. 들여쓰기 한 칸 차이가 실행 여부를 가릅니다.

주의 들여쓰기 오류 두 가지 시끄러운 오류 vs 조용한 오작동

정의 들여쓰기를 아예 안 하면 오류가 나서 바로 알지만, 일부만 어긋나면 오류 없이 결과만 틀립니다.

```
# ① 시끄러운 오류 — 바로 알아챌
if temp > 80:
    print("주의")          # IndentationError 발생

# ② 조용한 오작동 — 발견 어려움 ⚠
if temp > 80:
    print("주의")          # 조건부 (정상)
    print("점검 필요")     # ← 들여쓰기가 풀리면 항상 실행됨
```

왜 쓰나 ②번이 훨씬 위험합니다. 오류가 안 나니 코드는 돌아가는데 결과만 틀리거든요. "조건이 거짓인데 왜 출력되지?" 싶으면 들여쓰기부터 확인하세요.

조건문 `else` — 나머지 경우 거짓일 때 실행

정의 `else`는 `if` 조건이 거짓일 때 실행할 코드를 담습니다. 조건식을 따로 쓰지 않고(나머지 전부를 맡음), `if`와 `else` 중 **항상 하나만** 실행됩니다.

형태 `if` 조건식: 참일 때 `else:` 거짓일 때

```
score = 75
if score >= 60:          # 75 >= 60 → 참
    print("합격")        # 이 줄만 실행
else:
    print("불합격")      # 건너뛴
```

▶ 합격

왜 쓰나 `if`만 쓰면 "조건에 맞을 때만" 처리하고 나머지는 아무 일도 안 합니다. `else`를 붙이면 두 경우를 모두 처리하죠. 두 블록이 동시에 실행되는 일은 절대 없습니다.

응용 `if`만 두 번 쓰는 것과 `if-else`는 다릅니다. 아래 두 코드는 온도가 90일 때 결과가 같습니다.

```
temp = 90
# 1안 — if만 사용
if temp > 80:
    print("경고")
print("정상")          # ← if 밖이라 무조건 실행 (잘못!)
```

```
# 2안 — else 사용
if temp > 80:
    print("경고")
else:
    print("정상")      # ← 조건이 거짓일 때만
```

- ▶ [1안] 경고 / 정상 ← 둘 다 출력되어 이상함
- ▶ [2안] 경고 ← 하나만 출력, 올바른

자주 하는 실수

"둘 중 하나만" 나와야 하는데 둘 다 나온다면, else 대신 if 밖에 코드를 둔 것은 아닌지 확인하세요.

실습 1 — 나이로 성인 판정하기

```
age = int(input("나이를 입력하세요: "))
if age >= 19:
    print("성인입니다")
else:
    print("미성년자입니다")
```

- ▶ 19 입력 → 성인입니다
- ▶ 18 입력 → 미성년자입니다

개념

입력값은 항상 문자열이므로 int()로 감싸는 것을 잊지 마세요(4일차). 그리고 경계인 **19와 18**을 둘 다 넣어 정확히 갈리는지 확인하는 것이 이 실습의 핵심입니다.

STEP 3

elif — 갈림길이 셋 이상

순서가 곧 논리다

조건문 elif 중간 조건 추가

정의 갈림길이 셋 이상일 때 if와 else 사이에 조건을 추가합니다. 앞 조건이 거짓일 때만 다음 elif를 검사하며, 필요한 만큼 이어 붙일 수 있습니다.

형태 if 조건1: ... elif 조건2: ... else: ...

```
score = 82
if score >= 90:
    print("우수")      # 82 >= 90 → 거짓, 건너뛴
elif score >= 70:
    print("보통")      # 82 >= 70 → 참, 실행
else:
    print("미흡")      # 앞이 참이었으므로 검사조차 안 함
```

▶ 보통

왜 쓰나 if를 여러 번 쓰면 모든 조건을 다 검사하지만, elif는 ****처음 참인 것 하나만**** 실행하고 나머지는 건너뛵니다. 등급·구간 판정에 딱 맞습니다.

주의 elif 순서의 함정 오늘 가장 중요한 함정

정의 elif는 위에서부터 검사해 처음 참인 것에서 멈춥니다. 그래서 ****넓은 조건을 위에 두면 좁은 조건이 영영 실행되지 않습니다.****

```
score = 100
# x 잘못된 순서 — 넓은 조건이 위에
if score >= 50:
    print("우수")      # 100도 50 이상이라 여기서 걸림!
elif score >= 70:
    print("보통")      # 도달 불가
elif score >= 50:
    print("미흡")      # 도달 불가
```

▶ 우수 ← 100점인데 "미흡" 조건이 맨 위였다면 미흡이 나왔을 것

왜 쓰나 수업에서 든 예가 정확히 이것입니다. `score >= 50`을 맨 위에 두면 100점도 90점도 전부 첫 조건에서 걸려버립니다.

응용 해결책은 하나 — ****더 좁고 까다로운 조건(큰 값)을 위에, 넓은 조건을 아래에**** 배치합니다.

```
score = 100
# ✓ 올바른 순서 — 좁은 조건이 위에
if score >= 90:
    print("우수")      # 100 >= 90 → 참
elif score >= 70:
    print("보통")
elif score >= 50:
    print("미흡")
else:
    print("비상")
```

▶ 우수

주의

점검법: 가장 극단적인 값(만점·최저점)을 넣어보세요. 100점이 "우수"로, 0점이 "비상"으로 나오면 순서가 맞는 것입니다.

실습 2 — 설비 온도 상태 판정하기

```
temp = int(input("측정 온도를 입력하세요: "))
if temp > 80:
    print("위험")
elif temp > 60:
    print("주의")
else:
    print("정상")
```

- ▶ 85 → 위험
- ▶ 70 → 주의
- ▶ 50 → 정상

설비 적용

여기서도 순서가 핵심입니다. `temp > 60`을 위에 두면 85도가 "주의"로 나옵니다. **높은 기준부터 아래로** 배치하는 것이 구간 판정의 공식이에요.

STEP 4

논리 연산자와 중첩 if

조건을 엮어 복잡한 판단으로

연산자 and · or · not 조건 엮기 (3일차 복습)

정의 and는 둘 다 참일 때, or는 하나라도 참일 때 참. not은 참·거짓을 뒤집습니다.

```
temp, vib = 85, 3.5
print(temp > 80 and vib > 4.0)    # True and False → False
print(temp > 80 or vib > 4.0)     # True or False → True
print(not (3 == 5))               # not False → True
```

```
▶ False
▶ True
▶ True
```

왜 쓰나 and는 "모두 만족해야 통과"(로그인 검증), or는 "하나라도 걸리면 조치"(위험 신호 감지)에 씁니다. not은 괄호로 감싸서 쓰면 범위가 분명해집니다.

응용 파이썬은 범위 조건을 수학처럼 이어 쓸 수 있습니다. 두 방식은 완전히 같은 뜻이지만 아래쪽이 훨씬 읽기 쉽습니다.

```
temp = 45
# 방법1 - and로 두 조건 연결
if temp >= 20 and temp <= 60:
    print("정상 범위")

# 방법2 - 파이썬 특유의 연쇄 비교 (권장)
if 20 <= temp <= 60:
    print("정상 범위")
```

```
▶ 정상 범위
▶ 정상 범위
```

조건문 중첩 if - 조건 안의 조건 단계적 판정

정의 if 블록 안에 또 if를 넣어 1차로 거른 뒤 2차로 세부 판정을 합니다. 들여쓰기가 한 단계 더 깊어집니다.

```
temp, vib = 78, 3.5
if temp > 60:                                # 1차: 온도가 높은가
    if vib > 3.0:                            # 2차: 진동까지 높은가
        print("고온·고진동 → 위험")
    else:
        print("고온 → 주의")
else:
    print("정상")
```

▶ 고온·고진동 → 위험

왜 쓰나 "온도가 높은데, 그중에서도 진동까지 높다면"처럼 조건이 단계적으로 좁혀질 때 씁니다. 중첩 자체는 무한히 가능하지만, 깊어질수록 읽기 어려워 권장되지 않습니다.

응용 중첩이 깊어지면 elif 체인으로 평탄하게 펴 수 있습니다. 아래 두 코드는 같은 판정을 하지만 오른쪽이 읽기 쉽습니다.

```
# 중첩 방식 (교안)
if temp > 80 or vib > 4.0:
    print("위험: 즉시 정지")
else:
    if current > 60 and temp > 70:
        print("주의: 부하 점검")
    elif vib > 2.5:
        print("주의: 진동 관찰")
    else:
        print("정상")
```

```
# elif 평탄화 (실습 파일 방식) – 결과 동일
if temp > 80 or vib > 4.0:
    print("위험: 즉시 정지")
elif current > 60 and temp > 70:
    print("주의: 부하 점검")
elif vib > 2.5:
    print("주의: 진동 관찰")
else:
    print("정상")
```

▶ 78 / 3.2 / 52 → 주의: 진동 관찰 (두 방식 모두 동일)

개념

평탄화가 항상 가능한 건 아닙니다. 1차 조건 안에서 여러 줄을 실행해야 하거나 조건이 서로 독립적이면 중첩이 자연스럽습니다. "읽기 쉬운 쪽"을 고르세요.

실습 3 — 두 조건을 모두 만족해야 통과

```
correct_id = "admin"
correct_pw = "1234"          # ← 문자열로 저장하는 것이 중요
user_id = input("아이디: ")
user_pw = input("비밀번호: ")
if user_id == correct_id and user_pw == correct_pw:
    print("로그인 성공")
else:
    print("로그인 실패")
```

▶ admin / 1234 → 로그인 성공

▶ 그 외 → 로그인 실패

자주 하는 실수

비밀번호를 `correct_pw = 1234`처럼 **숫자로 저장하면 절대 로그인이 안 됩니다**. `input()`이 돌려주는 값은 항상 문자열이라 `"1234" == 1234`가 되어 늘 `False`거든요. 비교할 두 값의 **자료형을 맞추는 것**이 핵심입니다.

STEP 5

for 와 range

같은 일을 N번 되풀이하기

반복문 for 문의 기본 구조 N번 반복

정의 for 변수 in range(횟수): 아래 들여쓰는 코드를 지정한 횟수만큼 되풀이합니다.

형태 for i in range(횟수): 반복할 코드

```
for i in range(3):  
    print("안녕하세요")    # 3번 반복  
print("끝")               # 들여쓰기 밖 → 1번만
```

```
▶ 안녕하세요  
▶ 안녕하세요  
▶ 안녕하세요  
▶ 끝
```

왜 쓰나 같은 코드를 복사·붙여넣기 하는 대신 "이 코드를 N번 실행하라"고 한 번만 지시합니다. 데이터 수백·수천 개를 처리하는 모든 코드의 바탕이에요.

개념

반복 변수 i를 본문에서 쓰지 않아도 됩니다. 목적이 그냥 "3번 반복"이라면 i는 자리만 지키죠. 이럴 때 관례적으로 _(밑줄)를 쓰기도 합니다.

내장함수 range() – 시작·끝·증가값 반복할 숫자 만들기

정의 range(끝)은 0부터, range(시작, 끝)은 시작부터, range(시작, 끝, 증가)는 간격을 두고 숫자를 만듭니다.

****끝 값은 포함되지 않습니다.****

```
for i in range(5):      print(i)    # 0 1 2 3 4  
for i in range(1, 6):  print(i)    # 1 2 3 4 5  
for i in range(0, 10, 2): print(i)  # 0 2 4 6 8  
for i in range(5, 0, -1): print(i)  # 5 4 3 2 1 (역순)
```

```
▶ 0 1 2 3 4  
▶ 1 2 3 4 5  
▶ 0 2 4 6 8  
▶ 5 4 3 2 1
```

왜 쓰나 "끝 값 미포함"은 슬라이싱과 똑같은 규칙입니다(5일차). 그래서 1부터 N까지 돌려면 `range(1, N+1)`처럼 `**+1**`을 해줘야 합니다.

응용 증가값을 음수로 주면 역순이 됩니다. 단, 방향이 안 맞으면 한 번도 실행되지 않고 조용히 넘어갑니다.

```
# 방향이 맞지 않는 경우
for v in range(0, 10, -2):
    print(v)
print("반복 끝")

# 시작 0에서 -2씩 가면 끝 10에 영원히 도달 못 함
# → 오류 없이 0번 실행되고 지나감
```

▶ 반복 끝 ← 반복 본문은 한 번도 실행되지 않음

자주 하는 실수

"반복문이 아무것도 출력 안 한다"면 `range`의 방향(시작·끝·증가값 부호)이 맞는지 확인하세요. 오류가 안 나서 찾기 어려운 유형입니다.

실습 1 — range로 숫자 흐름 출력하기

```
n = int(input("끝 숫자 N을 입력하세요: "))
for i in range(1, n + 1):      # 1부터 N까지 (+1 주의)
    print(i)
for i in range(2, n + 1, 2):   # 짝수만
    print(i)
for i in range(n, 0, -1):      # N부터 1까지 역순
    print(i)
```

▶ N=5 → 1 2 3 4 5 / 짝수 2 4 / 역순 5 4 3 2 1

STEP 6

누적 변수 패턴

반복의 결과를 모으는 법

반복문은 그냥 돌기만 해서는 쓸모가 적습니다. 회차마다 나온 값을 **어딘가에 모아야** 합계·개수·평균이 나오죠. 그
그릇이 누적 변수입니다.

패턴 합계 패턴 `total += i`

정의 반복 밖에서 누적 변수를 0으로 준비하고, 반복 안에서 값을 더해 나갑니다.

```
total = 0                # 반복 밖에서 한 번만 초기화
for i in range(1, 6):    # i = 1,2,3,4,5
    total += i           # total = total + i
print("최종 합계:", total)
```

▶ 최종 합계: 15

왜 쓰나 6일차의 +=(복합 할당)가 여기서 진가를 발휘합니다. 값이 몇 개든 코드 길이는 그대로예요.

응용 한 회차씩 손으로 따라가면 원리가 확실히 잡힙니다.

회차	i	누적 전	누적 후
시작	—	—	0
1회차	1	0	1
2회차	2	1	3
3회차	3	3	6
4회차	4	6	10
5회차	5	10	15

주의 초기화 위치 — 밖이냐 안이냐 오늘 두 번째 함정

정의 누적 변수를 반복 **안**에서 0으로 만들면 매 회차 초기화되어 누적이 되지 않습니다.

```
# x 반복 안에서 초기화
for i in range(1, 6):
    total = 0          # ← 매번 0으로 되돌아감
    total += i
print(total)          # 5 (마지막 i만 남음)
```

```
# ✓ 반복 밖에서 초기화
total = 0
for i in range(1, 6):
    total += i
print(total)          # 15 (정상 누적)
```

```
▶ 5
▶ 15
```

왜 쓰나 수업 노트에서 이 차이를 직접 짚어보며 확인했습니다. "합계가 이상하게 작다"면 초기화가 반복 안에 들어갔는지 먼저 보세요.

패턴 개수 세기 · 평균 패턴 `count += 1 / total ÷ count`

정의 조건을 만족할 때만 1씩 더하면 개수, 합계를 개수로 나누면 평균입니다.

```
# 개수 세기
count = 0
for i in range(1, 11):
    if i > 5:          # 반복 안에 조건문 (오늘의 조합!)
        count += 1
print("개수:", count) # 5
```

```
# 평균
total, cnt = 0, 0
for i in range(1, 6):
    total += i; cnt += 1
if cnt > 0:           # 0으로 나누기 방지
    print("평균:", total / cnt)
```

```
▶ 개수: 5
▶ 평균: 3.0
```

왜 쓰나 `for` 안에 `if`를 넣는 이 조합이 오늘 배운 두 도구가 만나는 지점입니다. "여러 값을 훑으면서 조건에 맞는 것만 처리"가 데이터 분석의 기본 동작이에요.

응용 값을 세는 대신 리스트에 모으면 나중에 다시 쓸 수 있습니다. 6일차의 `append`와 결합한 패턴입니다.

```

over_list = []                # 빈 리스트 준비
for i in range(1, 11):
    if i > 5:
        over_list.append(i)   # 조건 맞는 값을 모으기
print(over_list)              # [6, 7, 8, 9, 10]
print("개수:", len(over_list))

```

▶ [6, 7, 8, 9, 10]

▶ 개수: 5

설비 적용

설비 응용(실습 4): 측정값을 n번 입력받아 80 초과 개수를 셉니다. 여기에 리스트를 더하면 "어떤 값이 초과였는지"까지 남길 수 있어 보고서에 바로 쓸 수 있습니다.

실습 2 · 4 — 합계와 초과 개수

```

# 실습 2 — 1부터 N까지 합계
n = int(input("N을 입력하세요: "))
total = 0
for i in range(1, n + 1):
    total += i
print("합:", total)

# 실습 4 — 기준(80) 초과 개수 세기
n = int(input("측정 횟수: "))
over_count = 0
for i in range(n):
    value = int(input("측정값: "))
    if value > 80:
        over_count += 1
print("초과 개수:", over_count)

```

▶ N=10 → 합: 55

▶ 5회 중 3개 초과 → 초과 개수: 3

STEP 7

enumerate 와 중첩 반복

인덱스가 필요할 때, 반복 속의 반복

내장함수 enumerate() 순서와 값을 함께

정의 리스트를 돌면서 **인덱스와 값을 동시에** 꺼냅니다. 이름 그대로 "날날이 세다"라는 뜻이에요.

형태 for 인덱스, 값 in enumerate(리스트):

```
temps = [33, 23, 45, 32, 28]

for t in enumerate(temps):
    print(t)          # (0, 33) 형태의 묶음으로 나옴

for idx, t in enumerate(temps):    # 변수 2개를 주면
    print(f"idx: {idx}, t: {t}")    # 각각 나눠서 받음
```

▶ (0, 33) (1, 23) (2, 45) (3, 32) (4, 28)
▶ idx: 0, t: 33
▶ idx: 1, t: 23
▶ ...

왜 쓰나 enumerate가 돌려주는 값은 (인덱스, 값) 두 개라, 받는 변수도 두 개를 준비해야 합니다. 하나만 쓰면 묶음 그대로 나와요.

응용 리스트를 도는 방법은 세 가지가 있고 목적에 따라 고릅니다.

```
temps = [33, 23, 45]

# ① 값만 필요할 때 — 가장 간단
for t in temps:
    print(t)

# ② 인덱스가 필요할 때 — range + len
for i in range(len(temps)):
    print(i, temps[i])

# ③ 둘 다 필요할 때 — enumerate (권장)
for i, t in enumerate(temps):
    print(i, t)
```

▶ ②와 ③은 같은 결과 — ③이 더 읽기 쉬움

개념

enumerate(리스트, 1)처럼 두 번째 인자를 주면 인덱스를 1부터 시작할 수 있습니다. "n번째 항목"처럼 사람이 읽는 번호를 붙일 때 편합니다.

반복문 중첩 반복 반복 안의 반복

정의 for 안에 또 for를 넣습니다. 바깥이 한 번 돌 때 안쪽이 전부 돌립니다.

```
for dan in range(2, 4):          # 바깥: 2단, 3단
    for su in range(1, 4):        # 안쪽: 1,2,3
        print(dan, "x", su, "=", dan * su)
    print("---")                  # 한 단이 끝날 때마다
```

```
▶ 2 x 1 = 2 / 2 x 2 = 4 / 2 x 3 = 6
▶ ---
▶ 3 x 1 = 3 / 3 x 2 = 6 / 3 x 3 = 9
▶ ---
```

왜 쓰나 구구단처럼 "바깥 값을 고정한 채 안쪽이 변하는" 구조에 씁니다. 바깥은 단수, 안쪽은 곱할 수를 담당하죠.

응용 중첩 반복은 실행 횟수가 ****곱셈****으로 늘어납니다. 데이터가 커질 때 속도를 좌우하는 부분이라 감을 잡아두세요.

```
# 바깥 x 안쪽 = 전체 실행 횟수
#   2 x 3   =    6
#   3 x 4   =   12
#   9 x 9   =   81
# 100 x 100 = 10,000 ← 데이터가 커지면 급격히 증가
```

주의

중첩 반복에 if를 더하면 "딱수 단만 출력" 같은 선택적 반복이 됩니다 — 오늘 배운 세 도구(for·for·if)가 한자리에 모이는 형태입니다.

실습 3 — 구구단과 2의 배수 단

```
# 전체 구구단 (2~9단)
for dan in range(2, 10):
    for su in range(1, 10):
        print(f"{dan} × {su} = {dan*su}")
    print("-----")

# 번외 — 2의 배수 단만 (range 증가값 활용)
for dan in range(2, 10, 2):      # 2, 4, 6, 8단
    print(f"\n{dan}단")
    for su in range(1, 10):
        print(f"{dan} X {su} = {dan*su}")
```

▶ $2 \times 1 = 2$... $2 \times 9 = 18$ / ----- / $3 \times 1 = 3$...

응용

"2의 배수 단만"을 만드는 방법이 두 가지입니다 — `range(2, 10, 2)`로 아예 짝수만 만들거나, `if dan % 2 == 0`으로 걸러내거나. 앞쪽이 더 효율적이지만(불필요한 반복 자체가 없음), 조건이 복잡하면 뒤쪽이 유연합니다.

치트시트

조건문 · 반복문 한 장 정리

A-1. 조건문

형태	하는 일	핵심 규칙
if 조건:	참일 때 실행	콜론 + 들여쓰기
else:	거짓일 때 실행	조건식 없음, 하나만 실행
elif 조건:	갈림길 셋 이상	좁은 조건을 위에
and / or / not	조건 엮기	둘 다 / 하나라도 / 뒤집기
20 <= x <= 60	범위 조건	연쇄 비교 (파이썬 특유)
if 안의 if	중첩 — 단계적 판정	깊어지면 elif로 평탄화

A-2. 반복문

형태	만들어지는 것	비고
range(5)	0 1 2 3 4	끝 값 미포함
range(1, 6)	1 2 3 4 5	1부터 N까지는 N+1
range(0, 10, 2)	0 2 4 6 8	증가값 지정
range(5, 0, -1)	5 4 3 2 1	역순 (음수 증가값)
for t in 리스트	값만	가장 간단
enumerate(리스트)	인덱스 + 값	변수 2개로 받기

A-3. 누적 변수 패턴 3종

목적	뼈대	주의
합계	total = 0 → total += i	초기화는 반복 밖에서
개수	count = 0 → if 조건: count += 1	조건을 만족할 때만
평균	total / count	count가 0인지 먼저 확인
모으기	lst = [] → lst.append(값)	값 자체를 남길 때

부록 B

오류·함정 사전

오늘 코드에서 걸려 넘어지기 쉬운 곳

X 이렇게 하면	무슨 일	✓ 바로잡기
if 다음 줄 들여쓰기 X	IndentationError	4칸(탭) 들여쓰기
넓은 조건을 elif 위에	아래 조건 도달 불가	좁은 조건(큰 값)부터
pw = 1234 (숫자)	input은 str → 항상 False	pw = "1234"
if x = 5	SyntaxError (대입)	if x == 5
for 안에 total = 0	매번 초기화 → 누적 안 됨	초기화는 반복 밖
range(0, 10, -2)	0번 실행 (조용히 통과)	방향·부호 확인
range(1, n) 1~n 기대	n이 빠짐	range(1, n+1)
list = [...]	내장 함수 이름 덮어씀	items = [...]

주의

마지막 항목을 눈여겨보세요. 변수 이름을 `list`로 지으면 파이썬 내장 함수 `list()`를 덮어서 나중에 그 함수를 못 씁니다. 2일차에 배운 "색이 입혀진 단어는 이름으로 쓰지 않는다"가 여기 적용돼요. `items·nums` 같은 이름을 쓰세요.

B-2. 수업 노트에서 찾은 오류

받아 적은 코드에 섞인 실수 두 개입니다. 직접 고쳐보면 개념이 더 확실해집니다.

```
# ① 4의 배수 누적 — 더하는 값이 잘못됨
total3 = 0
for i in range(1, 16):
    if i % 4 == 0:
        total3 += 3      # x 3을 더하고 있음 → 결과 9
        # total3 += i    # ✓ 값 자체(4, 8, 12)를 더해야 → 24
```

```
# ② 로그인 비교 — 자료형 불일치
user_password = 1234      # x 숫자
input_password = input("비밀번호: ") # str
# "1234" == 1234 → 항상 False, 로그인 영원히 실패
user_password = "1234"    # ✓ 문자열로 통일
```

- ▶ ① 잘못: 9 / 올바른: 24
- ▶ ② 자료형을 맞춰야 비교가 성립

B-3. 결과가 이상할 때 — 조건문 읽기 절차

오류는 안 나는데 결과만 틀릴 때가 조건문에서 가장 흔합니다. 이 순서로 짚으면 대부분 찾습니다.

순서	할 일	구체적으로
① 변수 값 확인	조건에 쓰인 변수의 현재 값 찾기	위쪽에서 어떤 값이 들어갔는지 print로 확인
② 조건 대입	위에서부터 넣어 처음 참인 조건 찾기	elif는 처음 참에서 멈춘다는 점 기억
③ 블록 추적	참인 블록의 들여쓴 코드만 따라가기	들여쓰기가 어긋나지 않았는지 확인

개념

실제 추적 예 — 기대한 "주의" 대신 "정상"이 나왔다면

- ① 증상 확인 — temp가 65인데 "정상"이 출력됨
- ② 값 점검 — 위쪽에서 temp에 정말 65가 들어갔는지 print로 확인 (입력을 int로 안 바꿨을 수도)
- ③ 원인 발견 — 조건 순서가 뒤집혔거나, 비교 기호 오타(80 대신 800)를 찾아냄

부록 C

두 도구가 만나는 곳 & 다음 시간

if + for = 데이터 필터링

오늘 배운 조건문과 반복문은 따로 쓸 때보다 **함께 쓸 때** 훨씬 강력합니다. 6일차의 리스트까지 더하면 진짜 데이터 처리가 시작됩니다.

세 도구의 조합 — 이상값만 골라내기

```
temps = [72, 85, 68, 91, 77, 83, 65]

over = []                                # 6일차: 빈 리스트
for t in temps:                          # 오늘: 반복
    if t > 80:                             # 오늘: 조건
        over.append(t)                    # 6일차: 값 모으기

print("초과 값:", over)
print("초과 개수:", len(over))
print(f"초과 비율: {len(over)/len(temps)*100:.1f}%")    # 5일차: f-string
```

```
▶ 초과 값: [85, 91, 83]
▶ 초과 개수: 3
▶ 초과 비율: 42.9%
```

설비 적용

이 여덟 줄이 사실상 **예지보전의 축소판**입니다 — 센서 값들을 훑으며(for), 기준을 넘는지 판정하고(if), 이상값을 모아(append), 비율까지 계산해 보고(f-string). 앞으로 배울 pandas는 이 과정을 더 빠르고 짧게 해주는 도구일 뿐, 사고방식은 똑같습니다.

369 게임 — 조건과 반복의 응용

실습 파일에 있던 369 게임은 오늘 배운 것들이 겹겹이 쌓인 좋은 예제입니다. 숫자를 문자열로 바꿔 한 글자씩 검사하죠 (5일차 문자열 + 오늘의 중첩 반복 + 조건).

```

for num in range(1, 21):
    s = str(num)                # 숫자를 문자열로 (4일차 형변환)
    count = 0
    for ch in s:                # 각 자리를 하나씩 (중첩 반복)
        if ch in "369":        # 3·6·9 포함? (5일차 in)
            count += 1         # 개수 세기 패턴
    if count > 0:
        print("짝" * count)    # 문자열 반복 (5일차)
    else:
        print(num)

```

▶ 1 2 짝 4 5 짝 7 8 짝 10 11 12 짝... (13은 짝, 33은 짝짝)

369 게임 — 숫자별로 따라가기

바깥 반복이 숫자 하나를 잡으면, 안쪽 반복이 그 숫자의 각 자리를 검사합니다. 표로 보면 흐름이 한눈에 들어옵니다.

숫자	문자열로	각 자리 검사	3·6·9 개수	출력
7	"7"	7 → 해당 없음	0	7
3	"3"	3 → 해당!	1	짝
13	"13"	1 → X, 3 → 해당!	1	짝
33	"33"	3 → 해당, 3 → 해당!	2	짝짝

다음 시간 예고

다음 주제	한 줄 미리보기
while 반복문	횟수가 아니라 "조건이 참인 동안" 반복 — 정답 맞출 때까지 계속
break · continue	반복 도중 빠져나오기 / 이번 회차만 건너뛰기
리스트 데이터 처리	조건 필터링 · 새 리스트 만들기 — 오늘 조합의 정식 버전

연결

교안 05_01의 Part 2·3이 다음 차례입니다. 오늘 for + if + append로 직접 만든 필터링 패턴이, 다음 시간엔 정식 기법으로 다시 등장합니다. 오늘 손으로 해봤기 때문에 훨씬 쉽게 느껴질 거예요.

맺음말

오늘의 성취와 다음 한 걸음

드디어 프로그램다워졌다

오늘로 코드가 처음으로 "판단하고 되풀이하는" 프로그램이 되었습니다. 3일차에 만들어만 두고 print로 보기만 했던 True/False가 실제 행동을 결정하게 됐고, 6일차의 리스트는 반복문과 만나 수백 개 값을 한 번에 처리할 수 있게 되었습니다. 조건문과 반복문은 앞으로 쓸 모든 코드의 뼈대라, 여기서 든 습관이 오래갑니다.

개념

복습 루틴 추천: ① `10.if.py`의 elif 순서 함정 예제를 직접 고쳐 100점이 "우수"로 나오게 만들기 → ② 누적 변수 초기화를 반복 안/밖으로 옮겨가며 결과가 15와 5로 갈리는 것 확인 → ③ `for + if + append`로 리스트에서 기준 초과 값만 뽑아내기 → ④ 중첩 반복으로 구구단을 출력하되 `if`를 더해 홀수 단만 출력해보기. 특히 **"조건 순서"**와 **"초기화의 위치"** 두 가지는 오류가 안 나면서 결과만 틀리는 유형이라, 직접 손으로 확인해두는 게 가장 확실합니다.

— 끝 —